

Backtesting — ทดสอบก่อนใช้เงินจริง

บทที่ 22–25: Vectorized · Event-Driven · Walk-Forward · Slippage & Fees

จบ Part นี้ → รู้ว่า strategy ที่ดูดีบนกระดาษ จะรอดในชีวิตจริงไหม

🔗 อ่าน Part นี้ยังไ้

● **มาถึงตรงนี้ได้แล้ว** — Part นี้สำคัญกว่าที่คิด: มันคือส่วนที่ตัดสินว่ากลยุทธ์ของคุณจริงหรือหลอกตัวเอง · ถ้าเวลาน้อยให้อ่าน **22.1 (lookahead bias)** กับ **บทที่ 25 (ต้นทุน)** ให้เข้าใจก่อน — สองเรื่องนี้ทำให้ backtest ส่วนใหญ่ในอินเทอร์เน็ตใช้ไม่ได้ · และดูตัวเลขติดลบในบทนี้ให้ดี มันตั้งใจให้ติดลบ

● **เคยเขียน backtest มาแล้ว** — ข้ามโครงสร้างพื้นฐานได้ · ที่ควรอ่าน: **22.1b (full-sample statistics)** ซึ่งเป็น lookahead ที่แนบเนียนกว่าการ shift(1) มาก · **บทที่ 24** ที่การแบ่ง train/test ผิดทำให้ "out-of-sample" ไม่ใช่ out-of-sample จริง · และตาราง cost sensitivity ในบทที่ 25 ที่ Sharpe ไหลจาก 1.90 → -1.06 ด้วยกลยุทธ์เดียวกันเป๊ะ

ทำไม BACKTESTING ถึงสำคัญมาก?

ก่อนใส่เงินจริงเข้าตลาด เราต้องรู้ว่า strategy ทำงานอย่างไรในอดีต — แต่ backtesting ที่ทำผิดวิธีอาจ **หลอกให้คิดว่า strategy ดี** ทั้งที่ไม่จริง:

- **Lookahead bias** — ใช้ข้อมูลอนาคตในการตัดสินใจวันนี้ → ผลตอบแทน "ปลอม" สูงเกินจริง
- **Overfitting** — ปรับ parameter จนดีกับข้อมูลอดีต แต่ล้มเหลวกับข้อมูลใหม่
- **Cost blindness** — ลืมคิด spread, commission, slippage ทำให้ P&L บิดเบือน

Part IV นี้จะสอนให้ backtest *อย่างถูกต้อง* — ด้วย vectorized, event-driven, walk-forward, และ realistic cost model

Running Example ตลอด Part IV — BTC Bybit/Binance Pair Strategy

เราจะต่อยอด PairStrategy และ TradingSystem จาก Part II/III มาทดสอบอย่างเป็นระบบ:

บท 22 → Vectorized backtest เร็ว · บท 23 → Event-driven สมจริง

บท 24 → Walk-forward ป้องกัน overfit · บท 25 → ใส่ต้นทุนจริง และดูว่า edge ยังอยู่ไหม

บทที่ 22: Vectorized Backtest

แก่นของบทนี้

Vectorized backtest ใช้ pandas/numpy คำนวณ signal และ P&L ทั้งหมดพร้อมกันโดยไม่มี for-loop — เร็วมาก เหมาะสำหรับ parameter sweep และ first-pass analysis กฎเหล็กข้อเดียว: `signal.shift(1)` ก่อนคุณ return เสมอ เพื่อหลีกเลี่ยง lookahead bias

แบบทดสอบก่อนเริ่ม — Lookahead Bias คืออะไร?

สมมติว่าคุณเขียนโค้ดแบบนี้:

```
signal = (zscore > 2.0).astype(int)
ret_strategy = signal * daily_return
```

? โค้ดนี้มี lookahead bias หรือเปล่า?

คำตอบ: มี — `signal[t]` คำนวณจาก zscore วัน `t` แต่ `daily_return[t]` = ราคาปิดวัน `t` / ราคาปิดวัน `t-1` ซึ่งคุณยังไม่รู้ตอน `t` เริ่มต้น
แก้ด้วย: `ret_strategy = signal.shift(1) * daily_return`
กฎเหล็ก: signal วัน `t` → execute วัน `t+1` เสมอ

22.1 Lookahead Bias — ศัตรูตัวร้ายของ Backtesting

Lookahead bias เกิดเมื่อ backtest ใช้ข้อมูล "อนาคต" ที่ trader จริงๆ ยังไม่มีในขณะตัดสินใจ ตัวอย่างที่พบบ่อยที่สุด:

```
# ตัวอย่างข้อมูลจำลอง BTC Bybit/Binance import numpy as np import pandas as pd np.random.seed(42) n = 500 dates =
pd.date_range("2023-01-01", periods=n, freq="D") bybit = 25000 + np.cumsum(np.random.normal(20, 300, n)) # ส่วน
ต่างสองตลาดต้อง "ติงกลับ" ได้ ไม่ใช่สุ่มลอย ๆ - ใช้ OU เหมือนบทที่ 12 # ถ้าใช้ noise เจย ๆ spread จะไม่มีแรงติงกลับ = กลยุทธ์นี้ไม่มี edge
ตั้งแต่ต้น kappa, sigma_ou = 0.15, 140.0 ou = np.zeros(n) for t in range(1, n): ou[t] = ou[t-1] + kappa * (0 - ou[t-
1]) + np.random.normal(0, sigma_ou) binance = bybit + ou # ราคาใกล้เคียง Bybit + spread ที่ mean-revert df =
pd.DataFrame({"bybit": bybit, "binance": binance}, index=dates) # คำนวณ z-score ของ spread WINDOW = 60 spread =
np.log(df["bybit"]) - np.log(df["binance"]) roll_mean = spread.rolling(WINDOW).mean() roll_std =
spread.rolling(WINDOW).std() zscore = (spread - roll_mean) / roll_std ENTRY_Z, EXIT_Z = 2.0, 0.5 # Raw signal
(ยังไม่ lag) raw_signal = pd.Series(0.0, index=df.index) raw_signal[zscore > ENTRY_Z] = -1.0 # short spread
raw_signal[zscore < -ENTRY_Z] = 1.0 # long spread # forward-fill: ถ้า position ต่อจนกว่า signal จะเปลี่ยน raw_signal
= raw_signal.replace(0, np.nan).ffill().fillna(0) #

# คำนวณ daily return ของ spread spread_return
= df["bybit"].pct_change() - df["binance"].pct_change() # ❌ ผิด - Lookahead Bias: signal วัน t คุณ return วัน t #
ราคา close ที่คำนวณ signal กับราคา que execute เป็นวันเดียวกัน bad_pnl = (raw_signal * spread_return).dropna() # ✅ ถูก -
shift(1): signal วัน t execute ที่ราคาวัน t+1 good_signal = raw_signal.shift(1).fillna(0) good_pnl =
(good_signal * spread_return).dropna() print(f"Same-bar (ไม่ shift) cumulative return: {(1+bad_pnl
).prod()-1:+.2%}") print(f"Shift(1) (ถูกต้อง) cumulative return: {(1+good_pnl).prod()-1:+.2%}") print(f"ส่วนต่าง:
{(1+bad_pnl).prod()-(1+good_pnl).prod():+.2%}")
```

► OUTPUT

```
Same-bar (ไม่ shift) cumulative return: +7.11%
Shift(1) (ถูกต้อง) cumulative return: +17.65%
ส่วนต่าง: -10.55%
```

😓 เอ๊ะ — ทำไมเวอร์ชัน "ผิด" กลับได้น้อยกว่า?

ผลออกมาสวนความคาดหมาย และนี่คือบทเรียนที่สำคัญกว่าตัวเลข: เหตุผล: signal ของเราเป็น **mean-reversion** — `raw_signal[t]` ถูกจุด
ชนวนโดยการเคลื่อนไหวที่เพิ่งเกิดขึ้นในวัน `t` เอง. พอเอาไปคูณ `spread_return[t]` ซึ่งคือการเคลื่อนไหวอันเดียวกันนั้น เราจึงกำลัง "เติมพินสวน

ทางหลังจากมันวิ่งไปแล้ว" = ขาดทุนที่แท่ง entry ทุกครั้ง

🚫 **สิ่งที่ต้องจำ:** ไม่ใช่ทุก lookahead ที่ทำให้ผลดูดีขึ้น — แต่ทุก lookahead ทำให้ผล **ไม่จริง** · อย่าใช้ "ผลดูดีเกินไป" เป็นเครื่องตรวจจับ lookahead เพราะบางแบบทำให้ผลดูแย่ลง แล้วคุณก็จะทิ้งกลยุทธ์ที่ดีไปโดยไม่รู้ตัว · เครื่องตรวจจับที่เชื่อถือได้มีอย่างเดียว: **ไล่ดูทุกบรรทัดว่าข้อมูลที่ใช้นั้น ณ เวลา t มีอยู่จริงตอน t หรือยัง**

22.1b Lookahead ตัวจริงที่หโลกคนได้ — full-sample statistics

แบบที่อันตรายกว่ามาก เพราะมันทำให้ผลดูดีขึ้นจริงและมองด้วยตาแทบไม่เห็น: ค่ารวม mean/std ของ z-score จากข้อมูลทั้งชุด แทนที่จะเป็น rolling window

```
# ❌ ผิด — ใช้ mean/std ของทั้งชุด: วันแรกของ backtest "รู้" ค่าเฉลี่ยของทั้งปีแล้ว z_full = (spread - spread.mean()) / spread.std() # ✅ ถูก — rolling: ใช้เฉพาะข้อมูลที่มีจริง ณ เวลานั้น z_roll = (spread - spread.rolling(WINDOW).mean()) / spread.rolling(WINDOW).std() def make_signal(z): s = pd.Series(0.0, index=df.index) s[z > ENTRY_Z] = -1.0 s[z < -ENTRY_Z] = 1.0 s[z.abs() < EXIT_Z] = 0.0 return s.replace(0, np.nan).ffill().fillna(0).shift(1).fillna(0) pnl_full = (make_signal(z_full) * spread_return).dropna() pnl_roll = (make_signal(z_roll) * spread_return).dropna() print(f"❌ full-sample z: {(1+pnl_full).prod()-1:+.2%}") print(f"✅ rolling z: {(1+pnl_roll).prod()-1:+.2%}") print(f"ghost profit: {(1+pnl_full).prod()-(1+pnl_roll).prod():+.2%}")
```

► OUTPUT

```
❌ full-sample z: +21.02%
✅ rolling z:      +17.65%
ghost profit:    +3.36%
```

🔴 **อ่านว่า:** spread.mean() ดูไร้พียงมาก — แต่มันคือค่าเฉลี่ยของทั้ง 500 วัน · การใช้มันตัดสินใจในวันที่ 61 เท่ากับบอกว่า "ตอนนั้นฉันรู้แล้วว่าอีก 439 วันข้างหน้าราคาเฉลี่ยจะเป็นเท่าไร" · ต่างจาก .rolling(60).mean() ที่มีย้อนหลังแค่ 60 วันที่ผ่านมาแล้วจริง ๆ

กฎเหล็ก 2 ข้อของ Vectorized Backtest

① shift(1) เสมอ — signal สร้างจากราคา close วันที่ t → execute ได้เร็วสุดที่ open วันที่ $t + 1$

② ทุกสถิติต้องเป็น rolling หรือ expanding เท่านั้น — เจอ .mean(), .std(), .min(), .max(), .quantile() ที่ไม่มี .rolling() หรือ .expanding() นำหน้า ให้สงสัยไว้ก่อนทุกครั้ง · รวมถึงการ normalize/scale ข้อมูลก่อนแบ่ง train-test ด้วย (เรื่องนี้ลงลึกใน Part V)

22.2 Vectorized Backtest ฉบับสมบูรณ์

```
def vectorized_backtest( price_a: pd.Series, price_b: pd.Series, window: int = 60, entry_z: float = 2.0, exit_z: float = 0.5, fee_rate: float = 0.001, ) -> pd.DataFrame: """ Vectorized pair trading backtest uu price series A และ B ส่งคืน DataFrame ที่มี: signal, net_return, equity, drawdown """ df = pd.DataFrame({"A": price_a, "B": price_b}) # 1. Spread และ z-score log_spread = np.log(df["A"]) - np.log(df["B"]) roll_mean = log_spread.rolling(window).mean() roll_std = log_spread.rolling(window).std() df["zscore"] = (log_spread - roll_mean) / roll_std # 2. Raw signal (ยังไม่ shift) raw = pd.Series(0.0, index=df.index) raw[df["zscore"] > entry_z] = -1.0 # spread กร้างเกิน → short raw[df["zscore"] < -entry_z] = 1.0 # spread แคบเกิน → long # ffill ถือ position — แต่ต้อง apply exit หลัง ffill เท่านั้น raw = raw.replace(0, np.nan).ffill().fillna(0) # ถือต่อจนกว่าจะ exit raw[df["zscore"].abs() < exit_z] = 0.0 # ปิด position หลัง ffill # 3. SHIFT(1) — ตัวใจสำคัญ df["signal"] = raw.shift(1).fillna(0) # 4. Returns ret_a = df["A"].pct_change() ret_b = df["B"].pct_change() # long spread = long A, short B → P&L = ret_A - ret_B gross_return = df["signal"] * (ret_a - ret_b) # 5. Transaction cost (เมื่อ position เปลี่ยน) position_change = df["signal"].diff().abs() # 1 เมื่อ flip/exit cost_per_bar = position_change * fee_rate * 2 # ทั้ง A และ B df["net_return"] = gross_return - cost_per_bar # 6. Equity curve (normalized: เริ่มที่ 1.0) df["equity"] = (1 + df["net_return"]).cumprod() # 7. Drawdown peak = df["equity"].cummax() df["drawdown"] = (df["equity"] - peak) / peak return df.dropna() # รัน backtest กับข้อมูล BTC จำลอง result = vectorized_backtest( price_a = pd.Series(bybit, index=dates, name="Bybit"), price_b = pd.Series(binance, index=dates, name="Binance"),
```

```
window=60, entry_z=2.0, exit_z=0.5, fee_rate=0.001, ) print(result[["zscore", "signal", "net_return", "equity", "drawdown"]].tail(6).round(5))
```

► OUTPUT

	zscore	signal	net_return	equity	drawdown
2024-05-09	0.95021	-1.0	-0.00150	0.90728	-0.10844
2024-05-10	1.24011	-1.0	-0.00252	0.90499	-0.11069
2024-05-11	1.15657	-1.0	0.00049	0.90544	-0.11025
2024-05-12	0.11836	-1.0	0.00850	0.91313	-0.10269
2024-05-13	-0.21545	0.0	-0.00200	0.91131	-0.10448
2024-05-14	0.07878	0.0	0.00000	0.91131	-0.10448

22.3 Performance Metrics: Sharpe, Calmar, Max Drawdown

เมื่อมี equity curve และ daily returns แล้ว เราคำนวณ metrics มาตรฐานได้ทันที:

```
def compute_metrics(result: pd.DataFrame, periods_per_year: int = 252) -> dict: """คำนวณ performance metrics
ครบชุดจาก backtest result""" rets = result["net_return"].dropna() eq = result["equity"].dropna() dd =
result["drawdown"].dropna() # Annualized return (compound) total_return = eq.iloc[-1] - 1.0 n_years = len(rets)
/ periods_per_year ann_return = (1 + total_return) ** (1 / max(n_years, 1e-9)) - 1 # Annualized volatility
ann_vol = rets.std() * np.sqrt(periods_per_year) # Sharpe Ratio (risk-free rate = 0 สำหรับ crypto) sharpe =
ann_return / ann_vol if ann_vol > 0 else 0.0 # Max Drawdown max_dd = float(dd.min()) # Calmar Ratio = annualized
return / |max drawdown| calmar = ann_return / abs(max_dd) if max_dd != 0 else 0.0 # Win Rate (บน active days
เท่านั้น) active = rets[rets != 0] win_rate = float((active > 0).mean()) if len(active) > 0 else 0.0 # Profit
Factor = gross profit / gross loss wins = active[active > 0].sum() losses = abs(active[active < 0]).sum()
profit_factor = wins / losses if losses > 0 else float("inf") # Average trade duration (consecutive non-zero
positions) pos = result["signal"] in_trade = (pos != 0).astype(int) avg_duration = (in_trade.groupby( (in_trade
!= in_trade.shift()).cumsum() ).sum().replace(0, np.nan).dropna().mean()) return { "Total Return (%)" :
round(total_return * 100, 2), "Ann. Return (%)" : round(ann_return * 100, 2), "Ann. Volatility (%)" :
round(ann_vol * 100, 2), "Sharpe Ratio" : round(sharpe, 3), "Max Drawdown (%)" : round(max_dd * 100, 2), "Calmar
Ratio" : round(calmar, 3), "Win Rate (%)" : round(win_rate * 100, 1), "Profit Factor" : round(profit_factor, 3),
"Avg Trade Duration" : round(float(avg_duration), 1), } m = compute_metrics(result) print("=" * 44) for k, v in
m.items(): print(f" {k:24s}: {v}")
```

► OUTPUT

```
=====
Total Return (%)      : -8.87
Ann. Return (%)       : -5.17
Ann. Volatility (%)   : 6.61
Sharpe Ratio          : -0.782
Max Drawdown (%)     : -11.86
Calmar Ratio          : -0.436
Win Rate (%)          : 40.1
Profit Factor         : 0.868
Avg Trade Duration    : 4.1
```

ตีความตัวเลข

Sharpe **-0.78** · Profit Factor **0.868** · Win Rate **40.1%** — กลยุทธ์นี้ขาดทุน และนั่นคือผลลัพธ์ที่ถูกต้องแล้ว ไม่ใช่โค้ดพัง

ก่อนหักค่าธรรมเนียม signal ชุดนี้ให้ **+17.65%** (เห็นในหัวข้อ 22.1) แต่พอใส่ fee เข้าไปกลับติดลบ — เพราะมันเทรดบ่อยเกินไปเมื่อเทียบกับขนาด edge ที่มี · นี่คือการขาดทุนของกลยุทธ์ส่วนใหญ่ที่ดูดีบนกระดาษ และเป็นเหตุผลว่าทำไมบทที่ 25 (Slippage, Fees & Reality) ถึงมีอยู่

$$\text{Sharpe} = \frac{R_{\text{ann}} - R_f}{\sigma_{\text{ann}}} \quad \text{Calmar} = \frac{R_{\text{ann}}}{|\text{MaxDD}|}$$

เป้าหมายสำหรับ live trading: **Sharpe > 1.0** และ **Calmar > 1.0** บน out-of-sample data — ตัวเลขข้างบนยังห่างไกลมาก

● [รอบสอง] ตัวเลขติดลบชุดนี้มีค่ามากกว่าที่คิด

ถ้าตัวอย่างในหนังสือให้ Sharpe 2.18 สวย ๆ คุณจะไม่มีวันได้เห็น **metrics** ของกลยุทธ์ที่ไม่เวิร์ค หน้าตาเป็นยังไง — และนั่นคือสิ่งที่ คุณจะเจอ บ่อยที่สุดในชีวิตจริง. จำ 3 สัญญาณนี้ไว้: **Profit Factor < 1.0** (กำไรรวมน้อยกว่าขาดทุนรวม) · **Calmar ติดลบ** (ขาดทุนพร้อมกับ drawdown ลึก) · **Win Rate 40% ที่ไม่ได้มาพร้อม payoff ratio สูง**

⚠ กับดักที่ตามมา: พอเห็นตัวเลขแบบนี้ สัญชาตญาณคือ "ลองปรับ window กับ entry_z ดู" — ระวังให้ดี เพราะการใส่เงื่อนไขพารามิเตอร์บนชุดข้อมูล เดียวกันจนกว่าจะเป็นบวก คือการ **overfit ที่แนบเนียนที่สุด**. บทที่ 24 (Walk-Forward) มีไว้เพื่อบอกว่าค่าที่ดูมานั้นจริงหรือแค่ฟลุค

22.4 Equity Curve และ Drawdown

```
def summarize_equity(result: pd.DataFrame) -> None: """แสดง equity curve summary แบบ text""" eq = result["equity"] dd = result["drawdown"] * 100 # Monthly returns monthly = (1 + result["net_return"]).resample("ME").prod() - 1 print("Monthly Returns (%):") print(" " + " ".join( f"{d.strftime('%b%y')}: {r*100:+5.1f}%" for d, r in monthly.items() )[:120]) print(f"\nEquity: start={eq.iloc[0]:.4f} " f"peak={eq.max():.4f} " f"end={eq.iloc[-1]:.4f}") print(f"Drawdown: worst={dd.min():.2f}% " f"avg_when_down={dd[dd<0].mean():.2f}%") # Time spent in drawdown pct_in_dd = (dd < -1).mean() * 100 print(f"Time in drawdown (>1%): {pct_in_dd:.1f}% of trading days") summarize_equity(result)
```

► OUTPUT

Monthly Returns (%):

Mar23: -0.4% Apr23: -2.3% May23: +0.8% Jun23: +1.2% Jul23: -4.3% Aug23: +0.6% Sep23: -0.8% Oct23: +0.6% Nov23: -0.1%
Equity: start=1.0000 peak=1.0176 end=0.9113
Drawdown: worst=-11.86% avg_when_down=-7.04%
Time in drawdown (>1%): 96.8% of trading days

ตัวอย่าง: Parameter Sweep แบบ Vectorized

ข้อดีของ vectorized คือสามารถ sweep parameter หลายร้อย combinations ได้ภายในไม่กี่วินาที

```
def parameter_sweep(price_a, price_b, windows, entry_zs, fee_rate=0.001): """Grid search บน window และ entry_z""" rows = [] for w in windows: for ez in entry_zs: res = vectorized_backtest( price_a, price_b, window=w, entry_z=ez, fee_rate=fee_rate ) if len(res) == 0: continue m = compute_metrics(res) rows.append( { "window": w, "entry_z": ez, "sharpe": m["Sharpe Ratio"], "calmar": m["Calmar Ratio"], "max_dd": m["Max Drawdown (%)"], "n_active": int((res["signal"] != 0).sum()), } ) df_out = pd.DataFrame(rows) return df_out.sort_values("sharpe", ascending=False).reset_index(drop=True) sweep = parameter_sweep( pd.Series(bybit, index=dates), pd.Series(binance, index=dates), windows = [30, 45, 60, 90, 120], entry_zs = [1.5, 2.0, 2.5, 3.0], ) print(sweep.head(8).to_string(index=False))
```

► OUTPUT

window	entry_z	sharpe	calmar	max_dd	n_active
120	3.0	0.862	0.941	-1.18	16
60	3.0	0.000	0.000	0.00	0
90	3.0	0.000	0.000	0.00	0
120	1.5	-0.430	-0.283	-9.40	222
90	2.0	-0.624	-0.276	-14.06	238
30	1.5	-0.662	-0.374	-12.28	302
120	2.0	-0.749	-0.407	-11.29	217
60	2.0	-0.782	-0.436	-11.86	268

► แบบฝึกหัด: เพิ่ม Sortino Ratio ใน compute_metrics

บทที่ 23: Event-Driven Backtest

แก่นของบทนี้

Event-driven backtest จำลองการซื้อขายจริงทีละ timestep แต่ละ bar ส่ง "event" ให้ระบบประมวลผล — จัดการ fills, partial fills, realistic order execution ได้ Realistic กว่า vectorized มาก ใช้ TradingSystem.step() จาก Part III เป็นหัวใจของ loop และแยก bar-data ออกจาก signal อย่างเคร่งครัด

23.1 ความต่างระหว่าง Vectorized และ Event-Driven

คุณสมบัติ	Vectorized	Event-Driven
ความเร็ว	เร็วมาก (numpy)	ช้ากว่า 10–100x
ความสมจริง	ต่ำ — ไม่มี partial fill	สูง — จำลองได้ละเอียด
Lookahead bias	ต้องระวัง shift(1) เอง	ป้องกันโดยโครงสร้าง loop
Order types	Market order เท่านั้น	Limit, Stop, IOC ได้
Portfolio state	ยุ่งยากถ้า multi-asset	จัดการง่ายกว่า
ใช้เมื่อ	Parameter sweep เบื้องต้น	Final validation ก่อน live

23.2 Data Structures สำหรับ Event System

```
from dataclasses import dataclass, field from typing import Optional, List from enum import Enum class OrderSide(Enum): LONG = "long" SHORT = "short" EXIT = "exit" @dataclass class BarData: """ข้อมูลราคาสำหรับ 1 timestep (1 bar)""" timestamp: pd.Timestamp symbol: str open_: float high: float low: float close: float volume: float = 0.0 @dataclass class Signal: """Output จาก strategy.step()""" timestamp: pd.Timestamp symbol: str direction: str # "LONG", "SHORT", "EXIT", "HOLD" strength: float = 1.0 @dataclass class Fill: """ผลการ execute order จริง""" timestamp: pd.Timestamp symbol: str side: str # "long" หรือ "short" quantity: float fill_price: float commission: float slippage: float @property def total_cost(self) -> float: return self.commission + self.slippage @dataclass class PortfolioState: """State ของ portfolio ณ เวลาหนึ่ง""" cash: float positions: dict = field(default_factory=dict) # symbol -> qty avg_prices: dict = field(default_factory=dict) # symbol -> avg cost realized_pnl: float = 0.0 unrealized_pnl: float = 0.0 def total_equity(self, current_prices: dict) -> float: unreal = sum( qty * current_prices.get(sym, self.avg_prices.get(sym, 0)) for sym, qty in self.positions.items() ) return self.cash + unreal
```

23.3 Event-Driven Loop ที่ใช้ TradingSystem.step()

```
class EventDrivenBacktester: """ Event-driven backtester ที่ใช้ TradingSystem จาก Part III ทุก bar -> MarketEvent -> Strategy.step() -> Signal -> Execution -> Fill """ def __init__(self, window: int = 60, entry_z: float = 2.0, exit_z: float = 0.5, fee_rate: float = 0.001, slippage_bps: float = 5.0, initial_capital: float = 1_000_000.0): self.window = window self.entry_z = entry_z self.exit_z = exit_z self.fee_rate = fee_rate self.slippage_bps = slippage_bps # basis points self.initial_capital = initial_capital # State self.cash = initial_capital self.position = 0.0 # +1 = long spread, -1 = short spread, 0 = flat self.entry_price = None self.bar_history: List[BarData] = [] # Records self.fills: List[Fill] = [] self.equity_curve: List[dict] = [] def _compute_zscore(self) -> Optional[float]: """คำนวณ z-score จาก bar history ปัจจุบัน""" if len(self.bar_history) < self.window: return None recent = self.bar_history[-self.window:] spreads = [np.log(b.close / b.volume) if b.volume > 0 else 0.0 for b in recent] # (volume ที่ใช้แทน price_b เพื่อความง่าย) arr = np.array(spreads) std = arr.std() return float((arr[-1] - arr.mean()) / std) if std > 0 else 0.0 def _compute_zscore_from_prices(self, closes_a: np.ndarray, closes_b: np.ndarray) -> float: """คำนวณ z-score จาก price arrays สองตัว""" if
```

```

len(closes_a) < self.window: return 0.0 window_a = closes_a[-self.window:] window_b = closes_b[-self.window:]
spread = np.log(window_a) - np.log(window_b) std = spread.std() return float((spread[-1] - spread.mean()) / std)
if std > 0 else 0.0 def _generate_signal(self, z: float) -> str: """แปลง z-score เป็น trading signal""" if z >
self.entry_z: return "SHORT" # spread กว้าง → short if z < -self.entry_z: return "LONG" # spread แคบ → long if
abs(z) < self.exit_z and self.position != 0: return "EXIT" # mean-reverted → ออก return "HOLD" def
_execute_signal(self, sig: str, price_a: float, price_b: float, ts: pd.Timestamp) -> Optional[Fill]: """Execute
signal จริง – คำนวณ fill price + cost""" if sig == "HOLD": return None if sig == "EXIT" and self.position == 0:
return None if sig in ("LONG", "SHORT") and self.position != 0: return None # มี position อยู่แล้ว # Slippage: ราคา
fill แยกว่า close เล็กน้อย slip_mult = self.slippage_bps / 10_000 if sig == "LONG": fill_a = price_a * (1 +
slip_mult) # buy A สูงกว่า close fill_b = price_b * (1 - slip_mult) # sell B ต่ำกว่า close elif sig == "SHORT":
fill_a = price_a * (1 - slip_mult) fill_b = price_b * (1 + slip_mult) else: # EXIT fill_a = price_a * (1 -
slip_mult if self.position > 0 else 1 + slip_mult) fill_b = price_b * (1 + slip_mult if self.position > 0 else 1
- slip_mult) # ขนาด position (10% ของ cash) notional = self.cash * 0.10 qty_a = notional / fill_a qty_b =
notional / fill_b commission = (qty_a * fill_a + qty_b * fill_b) * self.fee_rate slippage = (qty_a * abs(fill_a
- price_a) + qty_b * abs(fill_b - price_b)) # อัปเดต position state if sig == "LONG": self.position = 1.0
self.entry_price = (fill_a, fill_b, qty_a, qty_b) elif sig == "SHORT": self.position = -1.0 self.entry_price =
(fill_a, fill_b, qty_a, qty_b) else: # EXIT if self.entry_price is not None: ep_a, ep_b, qy_a, qy_b =
self.entry_price if self.position == 1.0: # was long: ขาย A, ซื้อ B pnl = (fill_a - ep_a) * qy_a - (fill_b - ep_b)
* qy_b else: # was short pnl = (ep_a - fill_a) * qy_a + (fill_b - ep_b) * qy_b self.cash += pnl - commission
self.position = 0.0 self.entry_price = None return Fill( timestamp = ts, symbol = "BTC-Pair", side =
sig.lower(), quantity = qty_a, fill_price = fill_a, commission = commission, slippage = slippage, ) def
run(self, df_a: pd.Series, df_b: pd.Series) -> pd.DataFrame: """ Main event loop df_a, df_b: price series ของ
asset A และ B (index = datetime) """ closes_a = df_a.values closes_b = df_b.values timestamps = df_a.index n =
len(closes_a) for i in range(n): ts = timestamps[i] price_a = float(closes_a[i]) price_b = float(closes_b[i]) #
— ขั้นตอนที่ 1: รับ MarketEvent ————— # (ใน live system: ไร data จาก exchange feed) # — ขั้นตอน
ตอนที่ 2: อัปเดต signal ————— # ใช้ข้อมูลถึง i (ไม่รวม i+1) → ไม่มี lookahead bias if i <
self.window: self.equity_curve.append({"ts": ts, "equity": self.cash}) continue z =
self._compute_zscore_from_prices(closes_a[:i], closes_b[:i]) sig = self._generate_signal(z) # — ขั้นตอนที่ 3:
Execute signal ————— fill = self._execute_signal(sig, price_a, price_b, ts) if fill
is not None: self.fills.append(fill) # — ขั้นตอนที่ 4: Mark-to-market equity ————— unreal = 0.0
if self.position != 0 and self.entry_price is not None: ep_a, ep_b, qy_a, qy_b = self.entry_price if
self.position == 1.0: unreal = (price_a - ep_a) * qy_a - (price_b - ep_b) * qy_b else: unreal = (ep_a - price_a)
* qy_a + (price_b - ep_b) * qy_b self.equity_curve.append({ "ts": ts, "equity": self.cash + unreal, "signal":
sig, "zscore": z, "position": self.position, }) return pd.DataFrame(self.equity_curve).set_index("ts") # — รับ
Event-Driven Backtest ————— bt_ed = EventDrivenBacktester( window=60, entry_z=2.0,
exit_z=0.5, fee_rate=0.001, slippage_bps=5.0, initial_capital=1_000_000, ) ed_result = bt_ed.run( df_a =
pd.Series(bybit, index=dates), df_b = pd.Series(binance, index=dates), ) print(f"Fills executed:
{len(bt_ed.fills)}") print(f"Final equity: {bt_ed.cash:,.0f}") print(f"Return: {bt_ed.cash /
bt_ed.initial_capital - 1:+.2%}") print(f"\nLast 3 fills:") for f in bt_ed.fills[-3:]: print(f"
{f.timestamp.date()} {f.side:5s} " f"qty={f.quantity:.4f} @ {f.fill_price:,.0f} " f"comm={f.commission:.2f}
slip={f.slippage:.2f}")

```

► OUTPUT

```

Fills executed: 28
Final equity:    1,014,251
Return:          +1.43%
Last 3 fills:
2024-03-22 exit  qty=2.7597 @ 36,687 comm=202.49 slip=101.25
2024-04-24 short qty=2.7385 @ 36,975 comm=202.52 slip=101.26
2024-05-03 exit  qty=2.8300 @ 35,780 comm=202.52 slip=101.26

```

23.4 ทำไม Event-Driven ให้ผลต่างจาก Vectorized

```

# เปรียบเทียบ vectorized vs event-driven บนข้อมูลเดียวกัน price_a_s = pd.Series(bybit, index=dates) price_b_s =
pd.Series(binance, index=dates) # Vectorized vec_result = vectorized_backtest(price_a_s, price_b_s, window=60,
entry_z=2.0, fee_rate=0.001) vec_metrics = compute_metrics(vec_result) # Event-driven ed_return = bt_ed.cash /
bt_ed.initial_capital - 1 print(f"=" * 50) print(f"{'Metric':<28} {'Vectorized':>10} {'Event-Driven':>12}")
print(f"-" * 50) print(f"{'Total Return (%)':<28} {vec_metrics['Total Return (%)']:>10.2f} " f"
{ed_return*100:>12.2f}") print(f"{'Sharpe Ratio':<28} {vec_metrics['Sharpe Ratio']:>10.3f} " f"{'(see

```

```
below')>12}")) print(f"{'Num fills':<28} {int((vec_result['signal'].diff()).abs(>0).sum()):>10} "
f"{len(bt_ed.fills):>12}")
```

► OUTPUT

```
=====
Metric                                Vectorized  Event-Driven
-----
Total Return (%)                      -8.87        1.43
Sharpe Ratio                          -0.782      (see below)
Num fills                             139         28
```

ทำไม Event-Driven ให้ผลต่ำกว่า?

- **Slippage:** Event-driven ใช้ fill price ที่ไม่ใช่ close price — ทำให้ P&L ลดลง
- **Signal timing:** Vectorized ใช้ข้อมูลทั้ง window คำนวณพร้อมกัน ส่วน event-driven คำนวณเฉพาะจาก data ที่มีถึง bar นั้น
- **Position management:** Vectorized ถือ fractional position ได้ ส่วน event-driven มีขนาด order เป็น discrete
- **ข้อสรุป:** Vectorized Sharpe > Event-Driven Sharpe เกือบทุกครั้ง — ใช้ event-driven เป็น "ground truth"

► แบบฝึกหัด: เพิ่ม stop-loss ใน EventDrivenBacktester

บทที่ 24: Walk-Forward Validation

แก่นของบทนี้

Walk-forward validation แบ่งข้อมูลตามเวลา — เทรน (in-sample) บน window แรก ทดสอบ (out-of-sample) บน window ถัดไป เลื่อนไปเรื่อยๆ วิธีนี้ป้องกัน overfitting และให้ผลที่ตรงกับ live trading มากที่สุด เพราะใช้ข้อมูลอนาคตในการ validate แทนที่จะ validate บนข้อมูลที่ train มาแล้ว

24.1 Overfitting Trap — Sharpe 2.0 กลายเป็นติดลบ

Overfitting เกิดเมื่อเรา optimize parameter บนข้อมูล *เดียวกัน* ที่ใช้ evaluate — parameter ที่ดีที่สุดใน "อดีต" ไม่ใช่ parameter ที่ดีที่สุดในอนาคต

```
# จำลอง overfitting: หา parameter ที่ดีที่สุดบนข้อมูลทั้งหมด # แล้ว evaluate บนข้อมูลเดียวกัน — นี่คือ in-sample performance
np.random.seed(0) n_total = 1000 dates_long = pd.date_range("2021-01-01", periods=n_total, freq="D") # สร้างข้อมูลที่
pair trading edge อ่อนมาก (จงใจ) btc_base = 30000 * np.exp(np.cumsum(np.random.normal(0.001, 0.03, n_total))) #
⚠️ ตรงนี้ "จงใจ" ให้ spread เป็น noise ล้วน — ไม่มี mean-reversion จริง # ต่างจากบทที่ 22 ที่ใช้ OU เพราะที่นั่นต้องการ edge จริงเพื่อ
โഴว backtest # แต่ตอนนี้ต้องการ edge "ปลอม" เพื่อให้เห็นว่า overfitting หน้าตาเป็นอย่างไร # → ถ้าข้อมูลมี edge จริง พารามิเตอร์ที่ดีในอดีต
ก็จะดีในอนาคตด้วย = ไม่ decay noise_scale = 120.0 bybit_l = btc_base + np.random.normal(0, noise_scale, n_total)
binance_l = btc_base + np.random.normal(0, noise_scale, n_total) pa = pd.Series(bybit_l, index=dates_long) pb =
pd.Series(binance_l, index=dates_long) # — แบ่งข้อมูลก่อนทำอะไรทั้งสิ้น — # ⚠️ จุดตายของการทดสอบแบบนี้: ถ้า optimize บน
ข้อมูล "ทั้งหมด" แล้วเอา # ท่อนท้ายมาเรียกว่า out-of-sample มันไม่ใช่ out-of-sample เลย # เพราะท่อนนั้นถูกใช้เลือก parameter ไปแล้ว →
decay ที่วัดได้ไร้ความหมาย train_a, train_b = pa.iloc[:800], pb.iloc[:800] test_a, test_b = pa.iloc[800:],
pb.iloc[800:] # In-sample: optimize บน 800 วันแรกเท่านั้น in_sample_sweep = parameter_sweep(train_a, train_b,
windows = [30, 60, 90, 120], entry_zs = [1.5, 2.0, 2.5, 3.0]) best_params = in_sample_sweep.iloc[0] print("Best
parameters (in-sample):") print(f" window={best_params['window']:.0f}, " f"entry_z=
{best_params['entry_z']:.1f}") print(f" In-sample Sharpe: {best_params['sharpe']:.2f}") # Out-of-sample:
parameter เดิม แต่ test บน 200 วันสุดท้ายที่ยังไม่เคยเห็น oos_res = vectorized_backtest( test_a, test_b, window =
int(best_params["window"]), entry_z = best_params["entry_z"], ) oos_m = compute_metrics(oos_res) print(f" Out-
of-sample Sharpe: {oos_m['Sharpe Ratio']:.2f}") kept = oos_m["Sharpe Ratio"] / best_params["sharpe"] print(f"
เหลือจาก in-sample: {kept:.1%}" f"{' ← ติดลบ = พลิกเป็นขาดทุน' if kept < 0 else ''}")
```

▶ OUTPUT

```
Best parameters (in-sample):
  window=120, entry_z=1.5
In-sample Sharpe: 1.99
Out-of-sample Sharpe: -0.28
เหลือจาก in-sample: -13.9% ← ติดลบ = พลิกเป็นขาดทุน
```

Overfitting เกิดจากอะไร?

เราทดสอบ 16 parameter combinations บนข้อมูลที่ **ไม่มี edge จริงเลย** — แต่ในบรรดา 16 ชุดนั้นย่อมมีชุดที่ "ดูดี" โดยบังเอิญ · การหยิบชุดที่ดีที่สุดมาใช้ = การหยิบ **ความบังเอิญที่แรงที่สุด** ไม่ใช่ edge · พอ data เปลี่ยน ความบังเอิญนั้นก็หายไป เหลือแค่ต้นทุนการเทรด จึงติดลบ

ยิ่งลองพารามิเตอร์มาก โอกาสเจอ "ความบังเอิญที่ดูดี" ยิ่งสูง — นี่คือเหตุผลที่จำนวนชุดที่ทดสอบต้องถูกนับเป็นต้นทุน (multiple testing) ไม่ใช่ของฟรี

📌 **อ่านว่า:** Sharpe 1.99 ในอดีต → -0.28 ในอนาคต · ไม่ได้แค่ "แย่งลง" แต่ **พลิกจากกำไรเป็นขาดทุน** ทั้งที่ใช้พารามิเตอร์ชุดเดียวกันเป๊ะ · และข้อมูลชุดนี้ **ไม่มี edge จริงเลยตั้งแต่ต้น** — Sharpe 1.99 ที่เห็นคือเสียงรบกวนที่เราไปคัดเลือกมาเองจากการลอง 16 ชุด

⚠️ **อีกกับดักในตาราง walk-forward ข้างล่าง — oos_sharpe = 0.000 ไม่ได้แปลว่า "เท่าทุน"**

หลาย fold จะได้ oos_sharpe = 0.000 พร้อม oos_return = 0.00 — นั่นแปลว่า **ไม่มีเทรดเลยในช่วงนั้น** ไม่ใช่ "เทรดแล้วเสมอตัว" · ถ้าเอาค่า 0 พวกนี้ไปหาค่าเฉลี่ยรวมกับ fold อื่น คุณจะได้นตัวเลขที่ดูนิ่งสวยงามโดยไม่มี ความหมาย

⚠️ **นิสัยที่ต้องติดตัว:** ดูจำนวนเทรดคู่กับ Sharpe เสมอ — Sharpe จาก 5 เทรดกับจาก 200 เทรดไม่ควรถูกอ่านด้วยสายตาเดียวกัน. ถ้า out-of-sample มีเทรดน้อยกว่า ~30 ครั้ง ให้ถือว่าตัวเลขนั้นเป็นสัญญาณรบกวนจนกว่าจะพิสูจน์เป็นอย่างอื่น

24.2 Walk-Forward ด้วย Expanding Window

Expanding window: in-sample เพิ่มขึ้นทุก fold แต่ out-of-sample เป็น fixed size เสมอ

Walk-Forward — Expanding Window

Fold 1: [TRAIN | TEST] Fold 2: [TRAIN | TEST] Fold 3: [TRAIN | TEST]

← expanding → ← fixed → ← fixed →

Walking Window (Rolling): Fold 1: [TRAIN | TEST] Fold 2: [TRAIN | TEST] Fold 3: [TRAIN | TEST]

← fixed window size → ← fixed →

```
from sklearn.model_selection import TimeSeriesSplit
def walk_forward_validation( price_a: pd.Series, price_b: pd.Series, param_grid: dict, n_splits: int = 5, test_size: int = 60, min_train: int = 120, expanding: bool = True, ) -> pd.DataFrame:
    """ Walk-forward validation ด้วย expanding หรือ rolling window ส่งคืน DataFrame ของ out-of-sample performance ทุก fold """
    assert len(price_a) == len(price_b), "price series ต้องมีความยาวเท่ากัน"
    n = len(price_a)
    folds = []
    # สร้าง fold indices
    fold_starts = list(range(min_train, n - test_size, test_size))
    for fold_idx, test_start in enumerate(fold_starts):
        test_end = min(test_start + test_size, n)
        if expanding:
            train_start = 0
        else:
            train_start = max(0, test_start - min_train)
        # rolling
        train_a = price_a.iloc[train_start:test_start]
        train_b = price_b.iloc[train_start:test_start]
        test_a = price_a.iloc[test_start:test_end]
        test_b = price_b.iloc[test_start:test_end]
        # — In-sample: หา best parameters
        best_sharpe = -np.inf
        best_params = None
        for w in param_grid["windows"]:
            for ez in param_grid["entry_zs"]:
                if len(train_a) <= w:
                    continue
                try:
                    res = vectorized_backtest( train_a, train_b, window=w, entry_z=ez, fee_rate=0.001 )
                    if len(res) == 0:
                        continue
                    m = compute_metrics(res)
                    if m["Sharpe Ratio"] > best_sharpe:
                        best_sharpe = m["Sharpe Ratio"]
                        best_params = {"window": w, "entry_z": ez}
                except Exception:
                    continue
        # — Out-of-sample: test ด้วย best parameters
        try:
            oos_res = vectorized_backtest( test_a, test_b, window = best_params["window"], entry_z = best_params["entry_z"], fee_rate = 0.001, )
            oos_m = compute_metrics(oos_res)
            if len(oos_res) > 0 else {}
        except Exception:
            oos_m = {}
        folds.append({ "fold": fold_idx + 1, "train_start": price_a.index[train_start].date(), "train_end": price_a.index[test_start - 1].date(), "test_start": price_a.index[test_start].date(), "test_end": price_a.index[test_end - 1].date(), "best_window": best_params["window"], "best_entry_z": best_params["entry_z"], "is_sharpe": round(best_sharpe, 3), "oos_sharpe": round(oos_m.get("Sharpe Ratio", 0), 3), "oos_return": round(oos_m.get("Total Return (%)", 0), 2), "oos_max_dd": round(oos_m.get("Max Drawdown (%)", 0), 2), })
    return pd.DataFrame(folds)
# — รัน Walk-Forward
wf_result = walk_forward_validation( price_a = pa, price_b = pb, param_grid = { "windows": [30, 60, 90, 120], "entry_zs": [1.5, 2.0, 2.5, 3.0], }, n_splits = 5, test_size = 100, min_train = 200, expanding = True, )
print(wf_result.to_string(index=False))
```

► OUTPUT

fold	train_start	train_end	test_start	test_end	best_window	best_entry_z	is_sharpe	oos_sharpe	oos_return	oos_max_dd
1	2021-01-01	2021-07-19	2021-07-20	2021-10-27	30	3.0	0.000	-1.132	-2.6	-4.1
2	2021-01-01	2021-10-27	2021-10-28	2022-02-04	120	2.0	0.530	0.000	0.0	0.0
3	2021-01-01	2022-02-04	2022-02-05	2022-05-15	120	2.0	0.409	0.000	0.0	0.0
4	2021-01-01	2022-05-15	2022-05-16	2022-08-23	120	1.5	0.728	0.000	0.0	0.0
5	2021-01-01	2022-08-23	2022-08-24	2022-12-01	120	1.5	0.845	0.000	0.0	0.0
6	2021-01-01	2022-12-01	2022-12-02	2023-03-11	120	1.5	2.239	0.000	0.0	0.0
7	2021-01-01	2023-03-11	2023-03-12	2023-06-19	120	1.5	1.985	0.000	0.0	0.0

24.3 วิเคราะห์ผล Walk-Forward

```
def analyze_walk_forward(wf: pd.DataFrame) -> None:
    """สรุปผล walk-forward validation"""
    print("=" * 56)
    print("Walk-Forward Summary")
    print("=" * 56)
    print(f"Folds tested: {len(wf)}")
    print(f"IS Sharpe (avg): {wf['is_sharpe'].mean():.3f} " f"± {wf['is_sharpe'].std():.3f}")
    print(f"OOS Sharpe (avg): {wf['oos_sharpe'].mean():.3f} " f"± {wf['oos_sharpe'].std():.3f}")
    ratio = wf["oos_sharpe"].mean() / wf["is_sharpe"].mean()
    print(f"OOS/IS Ratio: {ratio:.1%} " f"({'acceptable' if ratio > 0.5 else 'likely overfit'})")
    print(f"OOS Sharpe > 0: " f"{(wf['oos_sharpe'] > 0).sum()}/{len(wf)} folds")
    print(f"OOS Sharpe >
```

```
1: " f"{{(wf['oos_sharpe'] > 1.0).sum()}/{len(wf)} folds") print(f"OOS Return (avg):
{wf['oos_return'].mean():.2f}%") print(f"OOS Max DD (avg): {wf['oos_max_dd'].mean():.2f}%") print() # Stability:
parameter consistency print("Parameter Stability:") for p in ["best_window", "best_entry_z"]: counts =
wf[p].value_counts() most_common = counts.index[0] pct = counts.iloc[0] / len(wf) * 100 print(f" {p}: most
common = {most_common} ({pct:.0f}% of folds)") analyze_walk_forward(wf_result)
```

► OUTPUT

```
=====
Walk-Forward Summary
=====
Folds tested:          7
IS Sharpe (avg):       0.962 ± 0.833
OOS Sharpe (avg):      -0.162 ± 0.428
OOS/IS Ratio:          -16.8% (likely overfit)
OOS Sharpe > 0:        0/7 folds
OOS Sharpe > 1:        0/7 folds
OOS Return (avg):      -0.37%
OOS Max DD (avg):      -0.59%
Parameter Stability:
  best_window: most common = 120 (86% of folds)
  best_entry_z: most common = 1.5 (57% of folds)
```

ตีความ OOS/IS RATIO

OOS/IS Ratio = 19.5% หมายความว่า strategy "รักษา" ได้เพียง 20% ของ in-sample performance ในอนาคต — นี่เป็นสัญญาณ overfit ที่ชัดเจน

$$\text{OOS/IS Ratio} = \frac{\text{OOS Sharpe (avg)}}{\text{IS Sharpe (avg)}}$$

- > 70% = ดีมาก — strategy มี real edge
- 50–70% = ยอมรับได้ — ลอง live กับ small size
- < 50% = overfit สูง — ต้องลด complexity หรือเพิ่มข้อมูล
- < 0% = strategy เสีย money ใน OOS — อย่า live เด็ดขาด

ตัวอย่าง: TimeSeriesSplit จาก scikit-learn

sklearn มี TimeSeriesSplit ที่ทำ walk-forward ได้สะดวก ระวัง: ไม่รองรับ test_size โดยตรง ต้องเขียน wrapper

```
from sklearn.model_selection import TimeSeriesSplit def walk_forward_sklearn(price_a, price_b,
param_grid, n_splits=5): """ใช้ sklearn TimeSeriesSplit""" n = len(price_a) tscv =
TimeSeriesSplit(n_splits=n_splits) results = [] for fold_idx, (train_idx, test_idx) in
enumerate(tscv.split(price_a)): train_a = price_a.iloc[train_idx] train_b = price_b.iloc[train_idx]
test_a = price_a.iloc[test_idx] test_b = price_b.iloc[test_idx] # หา best params ใน train best_sharpe = -
np.inf best_p = None for w in param_grid["windows"]: for ez in param_grid["entry_zs"]: if len(train_a) <=
w: continue res = vectorized_backtest(train_a, train_b, window=w, entry_z=ez) if len(res) == 0: continue
sh = compute_metrics(res)["Sharpe Ratio"] if sh > best_sharpe: best_sharpe, best_p = sh, {"window": w,
"entry_z": ez} if best_p is None: continue # Test oos = vectorized_backtest(test_a, test_b, **best_p)
oos_m = compute_metrics(oos) if len(oos) > 0 else {} results.append({ "fold": fold_idx + 1, "is_sharpe":
round(best_sharpe, 3), "oos_sharpe": round(oos_m.get("Sharpe Ratio", 0), 3), "window": best_p["window"],
"entry_z": best_p["entry_z"], }) return pd.DataFrame(results) wf_sk = walk_forward_sklearn(
pa, pb, param_grid={"windows": [30, 60, 90], "entry_zs": [1.5, 2.0, 2.5]}, n_splits=5,
) print(wf_sk.to_string(index=False))
```

► แบบฝึกหัด: เปรียบเทียบ expanding vs rolling window

บทที่ 25: Slippage, Fees & Reality

แก่นของบทนี้

Strategy ที่ดูดีใน backtest บ่อยครั้ง "หาย" ไปเมื่อใส่ต้นทุนจริง: bid-ask spread, commission, market impact สูตรคร่าวๆ:

$$\text{cost per trade} = \frac{\text{spread}}{2} + \text{commission} + \text{market_impact}$$

บทนี้จะแสดงให้เห็นว่า strategy ที่ดูดี (Sharpe **1.90** เมื่อไม่คิด cost) เหลือ Sharpe **0.57** ที่ cost ต่ำ **0.00** ที่ cost ระดับกลาง และ **-1.06** ที่ cost สูง — กลยุทธ์เดียวกันเป๊ะ ต่างแค่ต้นทุน

25.1 ประเภทของ Transaction Costs

ประเภท	สาเหตุ	ขนาดปกติ (crypto)	สูตร
Bid-Ask Spread	Market maker กำไร	0.5–5 bps	spread/2 ต่อ side
Commission (Maker)	Exchange fee สำหรับ limit order	0–2 bps	notional × maker_rate
Commission (Taker)	Exchange fee สำหรับ market order	5–10 bps	notional × taker_rate
Market Impact	Order ขยับราคาตัวเอง	1–20 bps (แล้วแต่ size)	ขึ้นกับ order size / ADV
Funding Rate	ค่า perp futures ถือค้างคืน	0–3 bps / 8h	position × rate
Latency Cost	Adverse selection	ขึ้นกับ strategy	ยาก model

25.2 Transaction Cost Model ครบถ้วน

```
class TransactionCostModel: """ Model ต้นทุนการเทรดแบบสมจริง รวม: spread + commission + market impact + funding """
def __init__(self, spread_bps: float = 2.0, maker_fee_bps: float = 1.0, taker_fee_bps: float = 7.0,
    impact_coeff: float = 0.1, adv_usd: float = 1_000_000, funding_rate_bps: float = 1.0, funding_period_h: float = 8.0): """
    spread_bps: bid-ask spread ในหน่วย basis points maker_fee_bps: commission สำหรับ limit order (maker)
    taker_fee_bps: commission สำหรับ market order (taker) impact_coeff: coefficient ของ market impact model adv_usd:
    average daily volume (USD) funding_rate_bps: perpetual funding rate ต่อ period funding_period_h: funding period
    (ชั่วโมง) """
    self.spread_bps = spread_bps
    self.maker_fee_bps = maker_fee_bps
    self.taker_fee_bps = taker_fee_bps
    self.impact_coeff = impact_coeff
    self.adv_usd = adv_usd
    self.funding_rate_bps = funding_rate_bps
    self.funding_period_h = funding_period_h

    def spread_cost(self, notional: float) -> float: """ต้นทุนจาก bid-ask spread (ต่อ round trip)"""
    return notional * (self.spread_bps / 10_000)

    def commission(self, notional: float, order_type: str = "taker") -> float: """Commission ต่อ round trip (entry + exit)"""
    rate = (self.taker_fee_bps if order_type == "taker" else self.maker_fee_bps)
    return notional * (rate / 10_000) * 2 # x2: entry + exit

    def market_impact(self, notional: float, volatility: float = 0.02) -> float: """Square-root market impact model:
    impact = sigma * coeff * sqrt(order_size / ADV) เหมาะสำหรับ order ขนาดกลาง-ใหญ่ """
    participation = notional / self.adv_usd
    return notional * volatility * self.impact_coeff * np.sqrt(participation)

    def funding(self, notional: float, hold_hours: float = 24.0) -> float: """ต้นทุน funding สำหรับ perpetual futures"""
    periods = hold_hours / self.funding_period_h
    return notional * (self.funding_rate_bps / 10_000) * periods

    def total_cost(self, notional: float, order_type: str = "taker", volatility: float = 0.02, hold_hours: float = 24.0) -> dict:
    """รวมต้นทุนทั้งหมด"""
    sp = self.spread_cost(notional)
    cm = self.commission(notional, order_type)
    mi = self.market_impact(notional, volatility)
    fn = self.funding(notional, hold_hours)
    tot = sp + cm + mi + fn
    return {
        "spread": sp,
        "commission": cm,
        "market_impact": mi,
        "funding": fn,
        "total": tot,
        "total_bps": tot / notional * 10_000,
    }

    # ตัวอย่าง: เทรด BTC $100,000
    tcm = TransactionCostModel(
        spread_bps = 2.0,
        taker_fee_bps = 7.0,
        impact_coeff = 0.10,
        adv_usd = 500_000_000, # BTC มี volume สูงมาก
        funding_rate_bps = 1.0,
    )
    costs = tcm.total_cost(notional=100_000, order_type="taker", volatility=0.025, hold_hours=24)
    print("Transaction Costs for $100,000 BTC trade:")
    for k, v in costs.items():
        if k == "total_bps":
            print(f"{'TOTAL (bps)':<20}: {v:8.2f} bps")
        else:
            print(f"{'':<20}: {v:8.2f}")
```

► OUTPUT

Transaction Costs for \$100,000 BTC trade:

```
-----
spread          : $   20.00
commission       : $  140.00
market_impact    : $    3.54
funding          : $   30.00
total            : $  193.54
TOTAL (bps)      :   19.35 bps
```

25.3 Realistic P&L — "Strategy ดี" หลังหักต้นทุน

```
def backtest_with_realistic_costs( price_a: pd.Series, price_b: pd.Series, window: int = 60, entry_z: float =
2.0, exit_z: float = 0.5, cost_model: TransactionCostModel = None, notional: float = 100_000, hold_hours: float
= 24.0, ) -> pd.DataFrame: """ Vectorized backtest พร้อม realistic cost model """ if cost_model is None:
cost_model = TransactionCostModel() df = pd.DataFrame({"A": price_a, "B": price_b}) # Z-score log_spread =
np.log(df["A"]) - np.log(df["B"]) roll_mean = log_spread.rolling(window).mean() roll_std =
log_spread.rolling(window).std() df["zscore"] = (log_spread - roll_mean) / roll_std # Signal + lag raw =
pd.Series(0.0, index=df.index) raw[df["zscore"] > entry_z] = -1.0 raw[df["zscore"] < -entry_z] = 1.0
raw[df["zscore"].abs() < exit_z] = 0.0 raw = raw.replace(0, np.nan).ffill().fillna(0) df["signal"] =
raw.shift(1).fillna(0) # Daily volatility (rolling) daily_vol = log_spread.rolling(window).std() # Gross return
ret_a = df["A"].pct_change() ret_b = df["B"].pct_change() df["gross_return"] = df["signal"] * (ret_a - ret_b) #
Realistic cost per trade (เมื่อ position เปลี่ยน) position_change = df["signal"].diff().abs() cost_per_trade =
position_change * ( cost_model.total_cost( notional = notional, order_type = "taker", volatility =
float(daily_vol.mean()), hold_hours = hold_hours, )["total"] / notional # แปลงเป็น fraction ) # Funding cost (ทุก
วันที่มี position) daily_funding = (df["signal"].abs() * cost_model.funding_rate_bps / 10_000 * (24 /
cost_model.funding_period_h)) df["net_return"] = df["gross_return"] - cost_per_trade - daily_funding
df["equity_gross"] = (1 + df["gross_return"]).cumprod() df["equity_net"] = (1 + df["net_return"]).cumprod() # ตั้ง
ชื่อ "equity" ให้ตรงกับที่ compute_metrics() คาดหวัง - และเลือกใช้ # equity_net เพราะ metrics ต้องวัดจากเงินหลังหักต้นทุน ไม่ใช่ก่อน
หัก df["equity"] = df["equity_net"] peak_net = df["equity_net"].cummax() df["drawdown"] = (df["equity_net"] -
peak_net) / peak_net return df.dropna() # — เปรียบเทียบ: ไม่มี cost vs realistic cost ————— tcm_low =
TransactionCostModel( spread_bps=1.0, taker_fee_bps=5.0, impact_coeff=0.05, funding_rate_bps=0.5) # optimistic
tcm_mid = TransactionCostModel( spread_bps=2.0, taker_fee_bps=7.0, impact_coeff=0.10, funding_rate_bps=1.0) #
realistic tcm_high = TransactionCostModel( spread_bps=5.0, taker_fee_bps=10.0, impact_coeff=0.20,
funding_rate_bps=2.0) # pessimistic pa_s = pd.Series(bybit, index=dates) pb_s = pd.Series(binance, index=dates)
scenarios = { "No Costs": None, "Low Costs": tcm_low, "Mid Costs": tcm_mid, "High Costs": tcm_high, } print(f"
{'Scenario':<14} {'Sharpe':>8} {'Return%':>9} {'MaxDD%':>8} {'PF':>7}") print("-" * 52) for name, tcm in
scenarios.items(): if tcm is None: res = vectorized_backtest(pa_s, pb_s, window=60, entry_z=2.0, fee_rate=0)
else: res = backtest_with_realistic_costs( pa_s, pb_s, window=60, entry_z=2.0, cost_model=tcm, notional=100_000
) m = compute_metrics(res) print(f"{name:<14} {m['Sharpe Ratio']:>8.3f} " f"{m['Total Return (%)']:>9.2f} " f"
{m['Max Drawdown (%)']:>8.2f} " f"{m['Profit Factor']:>7.3f}")
```

► OUTPUT

Scenario	Sharpe	Return%	MaxDD%	PF
No Costs	1.903	22.07	-3.62	1.453
Low Costs	0.569	7.94	-5.43	1.101
Mid Costs	-0.006	-0.08	-6.31	1.005
High Costs	-1.057	-14.25	-16.68	0.842

ผลกระทบของ TRANSACTION COSTS

จาก Sharpe **1.90** (ไม่มี cost) → **-1.06** (high cost) — กลยุทธ์พลิกจากกำไรเป็นขาดทุน การเพิ่ม cost ทำให้:

1. **Return ลดลง** — โดยตรงจากค่าใช้จ่าย
2. **Sharpe ลดลงมากกว่า Return** — เพราะ cost เพิ่ม variance
3. **Max Drawdown แย่ลง** — cost ทำให้ drawdown นานขึ้นและลึกขึ้น

สูตรประมาณ break-even: $\text{trade freq} \times \text{cost/trade} < \text{gross alpha/day}$

25.4 Cost Sensitivity Table

```
def cost_sensitivity_table( price_a: pd.Series, price_b: pd.Series, window: int = 60, entry_z: float = 2.0,
spread_range: list = None, commission_range: list = None, ) -> pd.DataFrame: """ สร้าง sensitivity table: แถว =
spread, คอลัมน์ = commission ค่าที่แสดง = Sharpe Ratio """ if spread_range is None: spread_range = [0.5, 1.0, 2.0,
3.0, 5.0] if commission_range is None: commission_range = [0.0, 2.0, 5.0, 7.0, 10.0] rows = [] for spread in
spread_range: row = {"spread_bps": spread} for comm in commission_range: tcm = TransactionCostModel( spread_bps
= spread, taker_fee_bps = comm, impact_coeff = 0.10, funding_rate_bps = 1.0, ) res =
backtest_with_realistic_costs( price_a, price_b, window=window, entry_z=entry_z, cost_model=tcm,
notional=100_000, ) m = compute_metrics(res) if len(res) > 0 else {} row[f"comm_{comm:.0f}bps"] = round(
m.get("Sharpe Ratio", 0), 2 ) rows.append(row) return pd.DataFrame(rows).set_index("spread_bps") tbl =
cost_sensitivity_table(pa_s, pb_s, window=60, entry_z=2.0) print("Sharpe Ratio Sensitivity (rows=spread,
cols=commission):") print(tbl.to_string())
```

► OUTPUT

```
Sharpe Ratio Sensitivity (rows=spread, cols=commission):
      comm_0bps  comm_2bps  comm_5bps  comm_7bps  comm_10bps
spread_bps
0.5           0.16       0.12       0.06       0.01      -0.06
1.0           0.16       0.12       0.05       0.00      -0.06
2.0           0.15       0.10       0.04      -0.01      -0.07
3.0           0.14       0.09       0.03      -0.02      -0.08
5.0           0.12       0.07       0.00      -0.04      -0.10
```

อ่าน Sensitivity Table อย่างไร

เซลล์แต่ละช่องคือ Sharpe Ratio ที่ได้หากใช้ spread + commission ชุดนั้น

- สีเขียว (Sharpe > 1.0): strategy ยังมี edge แม้มี cost
- สีเหลือง (0.5–1.0): marginally profitable
- สีแดง (< 0.5): cost ทำลาย edge หมด

สำหรับ BTC pair ที่ Bybit/Binance: spread ปกติ 1–2 bps, taker fee 7 bps → Sharpe ประมาณ 1.2–1.5 ซึ่งยังดี แต่ถ้าใช้ market order บ่อย (spread 5 bps + comm 10 bps) → Sharpe เหลือ 0.06 ≈ flat

25.5 Market Impact Model — เมื่อ Order ใหญ่กระทบราคา

```
def market_impact_analysis(adv_usd: float = 500_000_000, vol: float = 0.025) -> None: """ แสดง market impact
เทียบกับ order size Square-root model: impact = vol * coeff * sqrt(Q/ADV) """ COEFF = 0.1 sizes = [10_000, 50_000,
100_000, 500_000, 1_000_000, 5_000_000, 10_000_000] print(f"Market Impact (ADV=${adv_usd/1e6:.0f}M, vol=
{vol:.1%})") print(f"{'Order Size':>12} {'Impact(bps)':>12} {'Impact($)':>12} " f"{'% of Order':>12}") print("-"
* 56) for q in sizes: impact_frac = vol * COEFF * np.sqrt(q / adv_usd) impact_bps = impact_frac * 10_000
impact_usd = impact_frac * q pct_of_order = impact_frac * 100 print(f"${q:>11,.0f} {impact_bps:>12.2f}
${impact_usd:>11,.2f} " f"{pct_of_order:>11.3f}%") market_impact_analysis(adv_usd=500_000_000, vol=0.025)
```

► OUTPUT

```
Market Impact (ADV=$500M, vol=2.5%)
  Order Size  Impact(bps)  Impact($)  % of Order
-----
$    10,000      0.11 $      0.11    0.001%
$    50,000      0.25 $     1.25    0.003%
$   100,000      0.35 $     3.54    0.004%
$   500,000      0.79 $    39.53    0.008%
```

\$ 1,000,000	1.12	\$ 111.80	0.011%
\$ 5,000,000	2.50	\$ 1,250.00	0.025%
\$ 10,000,000	3.54	\$ 3,535.53	0.035%

Square-Root Market Impact Law

สูตรที่ถูกใช้แพร่หลายที่สุดสำหรับ market impact:

$$\text{Impact} = \sigma \cdot \kappa \cdot \sqrt{\frac{Q}{\text{ADV}}}$$

โดย σ = daily vol, κ = impact coefficient (~0.1–0.5), Q = order size, ADV = average daily volume

Key insight: impact ขึ้นตาม *square root* ของ size — trade 4x ใหญ่ขึ้น → impact เพิ่มแค่ 2x

ตัวอย่าง: ประเมินว่า strategy ของเรา Scalable แค่ไหน

```
def max_scalable_aum(gross_sharpe: float, target_net_sharpe: float, adv_usd: float, trade_freq_yearly:
int, vol: float = 0.025, impact_coeff: float = 0.10, fixed_cost_bps: float = 10.0) -> float: """ หา AUM
สูงสุดที่ strategy ยังทำกำไรได้หลังหัก market impact คืนค่า max AUM (USD) """ # gross alpha per trade (ประมาณจาก
Sharpe) alpha_per_trade = gross_sharpe * vol / np.sqrt(trade_freq_yearly) # target net alpha (gross -
fixed_cost) fixed_cost_frac = fixed_cost_bps / 10_000 net_alpha = alpha_per_trade - fixed_cost_frac if
net_alpha <= 0: return 0.0 # fixed cost ทำลาย alpha ทั้งหมดแล้ว # หา max notional จาก: impact = net_alpha -
target_alpha # vol * coeff * sqrt(Q / ADV) = net_alpha - target_net_alpha_per_trade target_net_per_trade
= (target_net_sharpe * vol / np.sqrt(trade_freq_yearly)) impact_budget = net_alpha - target_net_per_trade
if impact_budget <= 0: return 0.0 max_participation = (impact_budget / (vol * impact_coeff)) ** 2
max_notional = max_participation * adv_usd return max_notional max_aum = max_scalable_aum( gross_sharpe =
2.18, target_net_sharpe = 1.0, adv_usd = 500_000_000, trade_freq_yearly = 50, fixed_cost_bps = 10.0, )
print(f"Max scalable AUM: ${max_aum:,.0f} ({max_aum/1e6:.1f}M USD)") print("ใส่ capital เกินนี้ → Sharpe ต่ำกว่า
1.0 เพราะ market impact")
```

► OUTPUT

Max scalable AUM: \$804,891,199 (804.9M USD)
ใส่ capital เกินนี้ → Sharpe ต่ำกว่า 1.0 เพราะ market impact

► แบบฝึกหัด: สร้าง cost scenario ที่ "worst case" และ plot equity curves

25.6 สรุป: Checklist ก่อน Live Trading

```
""" Checklist ก่อน deploy strategy ใด ๆ เข้า live trading """ CHECKLIST = { "1. No Lookahead Bias": [ "signal ทุกตัว
ถูก shift(1) ก่อน calculate P&L", "ไม่ใช้ข้อมูลอนาคตในทุก feature", "ตรวจสอบด้วย event-driven backtest ด้วย", ], "2.
Walk-Forward Passed": [ "OOS/IS ratio > 50%", "OOS Sharpe > 0 ทุก fold", "Parameter stable ข้ามหลาย fold", ], "3.
Realistic Costs": [ "ใส่ bid-ask spread ตามที่ตลาดจริงเป็น", "ใส่ commission ที่ exchange เรียกเก็บ (taker สำหรับ market
order)", "ใส่ market impact โดยเฉพาะถ้า order ใหญ่", "ใส่ funding rate สำหรับ perpetual futures", ], "4. Metrics ผ่าน
เกณฑ์ (OOS + Realistic Costs)": [ "Sharpe > 1.0", "Calmar > 1.0", "Max Drawdown < 15%", "Profit Factor > 1.2", ],
"5. Sanity Checks": [ "จำนวน trades มากพอ (> 30) เพื่อ statistical significance", "ทดสอบกับข้อมูลหลาย time periods",
"ไม่ overfitted ต่อ regime เดียว (เช่น bull market 2021)", ], } for category, items in CHECKLIST.items():
print(f"\n{category}") for item in items: print(f" □ {item}") print("\nถ้าผ่านทุกข้อ → เริ่ม paper trading 1-3 เดือน
ก่อน live")
```

► OUTPUT

1. No Lookahead Bias
 - signal ทุกตัวถูก shift(1) ก่อน calculate P&L
 - ไม่ใช้ข้อมูลอนาคตในทุก feature
 - ตรวจสอบด้วย event-driven backtest ด้วย
2. Walk-Forward Passed
 - OOS/IS ratio > 50%

- ❑ OOS Sharpe > 0 ทุก fold
- ❑ Parameter stable ข้ามหลาย fold

3. Realistic Costs

- ❑ ใส่ bid-ask spread ตามที่ตลาดจริงเป็น
- ❑ ใส่ commission ที่ exchange เรียกเก็บ (taker สำหรับ market order)
- ❑ ใส่ market impact โดยเฉพาะถ้า order ใหญ่
- ❑ ใส่ funding rate สำหรับ perpetual futures

4. Metrics ผ่านเกณฑ์ (OOS + Realistic Costs)

- ❑ Sharpe > 1.0
- ❑ Calmar > 1.0
- ❑ Max Drawdown < 15%
- ❑ Profit Factor > 1.2

5. Sanity Checks

- ❑ จำนวน trades มากพอ (> 30) เพื่อ statistical significance
- ❑ ทดสอบกับข้อมูลหลาย time periods
- ❑ ไม่ overfitted ต่อ regime เดียว (เช่น bull market 2021)

ถ้าผ่านทุกข้อ → เริ่ม paper trading 1-3 เดือนก่อน live

► **แบบฝึกหัด:** คำนวณ minimum edge ที่ต้องมีเพื่อ breakeven หลัง cost

จบ Part IV — สิ่งที่ได้จาก Backtesting

ก่อน Part IV: เทรน model ดูผลดี แต่ไม่รู้ว่ามี edge จริงหรือแค่ lucky/overfit

หลัง Part IV: มีกระบวนการที่เชื่อถือได้ก่อน deploy ทุกครั้ง

- **บท 22** — Vectorized backtest เร็วสำหรับ parameter sweep พร้อม shift(1) หยุด lookahead bias ✓
- **บท 23** — Event-driven backtest สมจริง จำลอง fill และ slippage ที่ละ bar ✓
- **บท 24** — Walk-forward validation เผย OOS/IS ratio และ parameter stability ✓
- **บท 25** — Realistic cost model (spread + commission + impact + funding) พร้อม sensitivity table ✓

ใน **Part V** เราจะนำ system ที่ผ่าน backtest แล้วไปต่อยอด: live data pipeline, order management, monitoring และ risk control แบบ production-grade

■ [สารบัญทั้งเล่ม](#) · [คำศัพท์ Quant Trading](#)